

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ярославский государственный технический университет»
Кафедра «Кибернетика»

Отчет по практике
защищен с оценкой _____
Руководитель,
зав. кафедрой, канд. тех. наук
_____ И. В. Тюкин
«__» _____ 2026г.

**СРАВНИТЕЛЬНЫЙ АНАЛИЗ И ОЦЕНКА
ПРОИЗВОДИТЕЛЬНОСТИ МЕТОДОВ СЕГМЕНТАЦИИ
ОБЪЕКТОВ ДЛЯ ОБНАРУЖЕНИЯ НАВИГАЦИОННЫХ
ОРИЕНТИРОВ БПЛА В УСЛОВИЯХ ОТСУТСТВИЯ GPS НА
ОСНОВЕ ДАТАСЕТА MIDAIR.**

Пояснительная записка
к практике о научно исследовательской работе

ЯГТУ 27.04.04 – 002 НИР

Отчет выполнил
студент группы ЦМИИ-20м
_____ М. Е. Колобаев
«__» _____ 2026г.

Содержание

1 Введение	3
2 Цели и задачи работы	5
3. Общая архитектура системы навигации.....	7
4. Датасет MidAir	9
5. Подготовка обучающего датасета.....	11
6. Обучение моделей сегментации.....	15
7. Результаты экспериментов.....	18
8. Анализ и сравнение моделей	31
9 Ожидаемые результаты проекта.....	Ошибка! Закладка не определена.
10 Перспективы развития проекта	35
11 Выводы и заключение	40
Список используемой литературы	42
Приложение А	44
Приложение Б.....	47
Приложение В	50
Приложение Г	51

1 Введение

Беспилотные летательные аппараты (БПЛА) в последние годы стали одним из наиболее динамично развивающихся направлений в области робототехники, авиации и искусственного интеллекта. Их применение охватывает широкий спектр задач: от картографирования местности и мониторинга инфраструктуры до проведения поисково-спасательных операций, точного сельского хозяйства, доставки грузов и решения военно-тактических задач. Стремительный рост числа сценариев эксплуатации БПЛА сопровождается повышением требований к надёжности и автономности этих систем.

Одним из фундаментальных условий функционирования современного БПЛА является наличие надёжного сигнала системы глобального позиционирования (GPS/GNSS). Именно GPS обеспечивает определение координат воздушного судна, построение маршрута и удержание заданного курса в пространстве. Однако на практике существует целый ряд факторов, способных нарушить устойчивый приём навигационного сигнала или полностью лишить аппарат доступа к нему.

К основным причинам потери GPS-навигации относятся: полёты в условиях плотной городской застройки или горного рельефа, создающих зоны радиотени; эксплуатация внутри зданий и в подземных сооружениях; целенаправленное глушение (джаммирование) и спуфинг (подмена) навигационного сигнала со стороны злоумышленников или в условиях радиоэлектронной борьбы; неблагоприятные атмосферные условия, а также технические сбои навигационного оборудования. В каждом из этих случаев аппарат, полностью зависящий от спутниковой навигации, оказывается либо обездвиженным, либо переходит в неуправляемый режим полёта, что представляет серьёзную угрозу безопасности.

Данное обстоятельство обуславливает актуальность разработки систем автономной навигации БПЛА, способных функционировать в условиях отсутствия или ненадёжности спутникового сигнала. Такие системы должны обеспечивать определение местоположения аппарата, оценку его скорости и ориентации на основе альтернативных источников информации — бортовых сенсоров: камер, инерциальных измерительных блоков (IMU), барометрических высотомеров, лидаров и радаров.

Среди всего многообразия подходов к беспутниковой навигации особое место занимает концепция навигации по визуальным ориентирам окружающей среды. Данный подход основан на том принципе, что природный или рукотворный ландшафт содержит устойчивые, хорошо различимые объекты — деревья, скальные образования, реки, строения, перекрёстки дорог, — которые могут служить надёжными точками привязки при определении положения аппарата в пространстве. Если бортовая система способна идентифицировать такие ориентиры на изображениях, получаемых с камеры,

и сопоставить их с предварительно сформированной картой, задача локализации БПЛА может быть решена без обращения к GPS.

Развитие глубокого обучения и появление высокопроизводительных моделей семантической сегментации и детектирования объектов открыло принципиально новые возможности для реализации этого подхода. Современные нейросетевые архитектуры семейства YOLO (You Only Look Once) позволяют в режиме реального времени обнаруживать и сегментировать объекты на изображениях, получаемых с видеокамер БПЛА. Это создаёт технологическую базу для построения интеллектуальной системы навигации по визуальным ориентирам.

Вместе с тем использование только визуальной информации сопряжено с рядом ограничений: резкие манёвры аппарата, размытие изображения при высокой скорости, ухудшение видимости в сложных атмосферных условиях (туман, осадки, сильная засветка) могут снизить качество детектирования ориентиров. Для компенсации этих недостатков целесообразно объединить визуальную навигацию с инерциальной системой измерений. Инерциальный измерительный блок (IMU), фиксирующий линейные ускорения и угловые скорости, позволяет интегрировать данные о движении аппарата между моментами надёжного обнаружения ориентиров, обеспечивая непрерывную оценку траектории.

Синтез методов компьютерного зрения, глубокого обучения и инерциальной навигации представляет собой перспективное направление исследований в области автономных интеллектуальных систем. Предлагаемая в данной работе система призвана продемонстрировать практическую состоятельность подобного мультимодального подхода применительно к задаче навигации БПЛА в условиях потери GPS-сигнала.

Для обучения и тестирования разрабатываемых компонентов системы был выбран датасет MidAir — синтетический набор данных, специально созданный для задач визуальной одометрии, глубинного зрения и навигации БПЛА. Датасет обеспечивает наличие точных ground truth траекторий, семантической разметки сцен, данных IMU и GPS, что делает его идеальным инструментом для обучения и оценки предлагаемых алгоритмов.

2 Цели и задачи работы

2.1 Объект и предмет исследования

Объектом исследования является процесс автономной навигации беспилотного летательного аппарата в условиях отсутствия сигнала глобальной навигационной спутниковой системы (ГНСС/GPS).

Предметом исследования служат методы и алгоритмы нейросетевой сегментации изображений для обнаружения и классификации визуальных ориентиров окружающей среды, а также характеристики производительности и точности соответствующих моделей применительно к задаче навигации БПЛА в реальном времени.

2.2 Цели и задачи работы

Целью данной работы является разработка и экспериментальная оценка системы сегментации объектов на изображении для автономной навигации беспилотного летательного аппарата, обеспечивающей определение местоположения в условиях отсутствия внешних навигационных сигналов путём интеграции методов визуальной сегментации ориентиров и инерциальных измерений.

Для достижения поставленной цели в рамках данной работы решаются следующие задачи:

- разработка общей архитектуры системы навигации, включающей модуль визуальной сегментации, формирование информационного вектора сцены и модуль инерциальной поддержки;
- подготовка обучающего датасета на основе семантических карт датасета MidAir путём автоматической конвертации в форматы YOLO-segmentation и SegFormer;
- обучение шести моделей instance segmentation архитектур YOLOv8 и YOLO11 в вариантах nano, small и medium;
- обучение трёх моделей semantic segmentation архитектуры SegFormer (B0, B1, B2) для сравнительного анализа;
- сравнительная оценка обученных моделей обоих классов по метрикам точности (Precision, Recall, F1, mAP50, mAP50-95, mIoU) и производительности (FPS, время инференса) как на GPU, так и на бортовом одноплатном компьютере Raspberry Pi 5;
- оценка потенциала оптимизации модели с помощью форматов ONNX и NCNN для применения на ограниченных вычислительных платформах;
- выбор оптимальной архитектуры для интеграции в систему навигации БПЛА реального времени.

2.3 Гипотеза исследования

Предполагается, что модели instance segmentation семейства YOLO, несмотря на меньшие значения попиксельных метрик по сравнению с моделями semantic segmentation (SegFormer), обеспечивают более полное и

пригодное для навигации описание сцены за счёт предоставления пространственных координат отдельных экземпляров ориентиров. При этом применение оптимизированных форматов инференса (ONNX, NCNN) позволит достичь производительности, приемлемой для практического применения на одноплатных компьютерах класса Raspberry Pi без использования специализированных нейровычислительных ускорителей.

3. Общая архитектура системы навигации

Предлагаемая система автономной навигации БПЛА при потере внешней навигации представляет собой многокомпонентный программно-аппаратный комплекс, объединяющий методы компьютерного зрения, машинного обучения и инерциальной навигации. Архитектура системы разработана с учётом требований реального времени, ограниченных вычислительных ресурсов бортового компьютера БПЛА и необходимости обеспечения надёжности в условиях неблагоприятной внешней среды.

Система включает следующие взаимосвязанные компоненты:

3.1. Камера БПЛА

Модуль получения визуальной информации — основной источник данных об окружающей среде. Бортовая камера обеспечивает непрерывный поток RGB-изображений, которые передаются в модуль нейросетевой обработки. К камере предъявляются требования высокой частоты кадров (не менее 25 FPS), широкого угла обзора и устойчивости к вибрациям, неизбежным в условиях полёта.

3.2. Нейросетевая модель сегментации

Ключевой интеллектуальный компонент системы. На каждом кадре, поступающем с камеры, модель выполняет семантическую сегментацию — попиксельную классификацию объектов сцены с формированием полигональных масок. В данной работе рассматриваются три класса визуальных ориентиров: деревья (tree), скальные образования (rock) и водные объекты (river). Выбор этих классов обусловлен их высокой визуальной устойчивостью, хорошей различимостью на аэрофотоснимках и широким распространением в природных ландшафтах — типичных зонах эксплуатации БПЛА.

3.3. Информационный вектор сцены

На основании результатов сегментации в дальнейшем будет формироваться компактный информационный вектор, описывающий текущую наблюдаемую сцену. Данный вектор будет кодировать типы обнаруженных ориентиров, их пространственное расположение в кадре, площадь масок и взаимное расположение объектов. Информационный вектор является унифицированным представлением сцены для последующего сравнения с картой ориентиров.

3.4. Модуль сопоставления сцены

Данный модуль, в дальнейшем, реализует процедуру сравнения текущего информационного вектора сцены с заблаговременно подготовленной картой ориентиров — базой данных, содержащей информационные векторы для различных точек известного маршрута или

территории. По результатам сопоставления будет производиться оценка местоположения БПЛА — будет определяться, какому узлу карты наиболее соответствует текущее наблюдение.

3.5. Инерциальная навигация (IMU)

Инерциальный измерительный блок обеспечивает непрерывное измерение линейных ускорений и угловых скоростей БПЛА. Данные IMU будут использоваться для интеграции траектории в промежутках между кадрами, в которых успешно обнаружены ориентиры, что позволяет сохранять актуальную оценку положения аппарата даже при временной потере визуальных ориентиров вследствие обструкций или резких манёвров. Объединение данных IMU и визуальной локализации будет реализовано посредством алгоритма фильтрации, обеспечивающего оптимальную оценку состояния системы.

4. Датасет MidAir

Для обучения нейросетевых моделей сегментации и валидации компонентов системы навигации был использован открытый синтетический датасет MidAir (Mid-Air Dataset). Датасет был разработан специально для задач визуальной одометрии, навигации БПЛА и оценки глубины в условиях разнообразных природных сцен и динамических сред.

4.1. Общая характеристика датасета

MidAir представляет собой сбор синтетических полётных траекторий БПЛА, сгенерированных в виртуальных природных сценах с использованием фотореалистичного рендеринга. Синтетическое происхождение данных обеспечивает несколько ключевых преимуществ: наличие точных ground truth аннотаций для всех модальностей данных, гарантированное соответствие между различными типами сенсорных данных, отсутствие ошибок разметки и возможность генерации данных для произвольных условий. [1]

Датасет включает следующие типы данных:

- RGB-изображения высокого разрешения, полученные со стереокамеры и монокамеры, закреплённой на симулированном БПЛА;
- карты глубины (depth maps) с точными значениями расстояния до объектов сцены для каждого пикселя изображения;
- семантические карты сегментации (semantic segmentation maps) с попиксельной разметкой принадлежности к классам объектов сцены;
- измерения инерциального блока (IMU): линейные ускорения по трём осям и угловые скорости (гироскоп);
- данные GPS: координаты БПЛА в трёхмерном пространстве с высокой точностью;
- траектории ground truth: эталонные пространственные траектории движения аппарата.

4.2. Используемые данные для визуальной навигации

В рамках данной работы для решения задачи визуальной навигации были задействованы два типа данных из датасета MidAir:

- RGB-изображения — непосредственно используются как входные данные для нейросетевой модели сегментации в режиме эксплуатации системы;
- семантические карты сегментации — применялись исключительно на этапе подготовки обучающего датасета для формирования эталонных масок ориентиров.

Семантические карты MidAir содержат пиксельную разметку объектов сцены в соответствии с предустановленной палитрой классов, включающей разнообразные элементы природного ландшафта: растительность различных видов, скальные породы, водные объекты, небо, почву и другие. Наличие детализированной семантической разметки позволило автоматически

сформировать обучающий датасет без необходимости ручной аннотации изображений.

4.3. Классы визуальных ориентиров

На основе анализа содержимого датасета MidAir и требований к навигационным ориентирам были выбраны три целевых класса объектов:

- tree (деревья) — растительные объекты, характеризующиеся высокой степенью присутствия в природных сценах, хорошей визуальной различимостью и пространственной стабильностью;
- rock (скальные объекты) — геологические образования, обладающие высокой устойчивостью формы и характерной текстурой, что делает их надёжными долгосрочными ориентирами;
- river (водные объекты) — реки и водоёмы, являющиеся легкоузнаваемыми линейными или площадными ориентирами, хорошо выделяемыми по яркости и текстурным признакам.

Перечисленные объекты были выбраны исходя из нескольких критериев: высокая встречаемость в природных сценах датасета MidAir, выраженная визуальная отличимость от фона, инвариантность к сезонным изменениям (особенно для скальных образований и водных объектов), а также геопространственная стабильность — ориентиры не перемещаются со временем и могут быть нанесены на долгосрочную карту.

5. Подготовка обучающего датасета

Для обучения моделей сегментации семейства YOLO требуется специфический формат разметки, отличный от формата семантических карт MidAir. В связи с этим был разработан автоматизированный конвейер конвертации сегментационных карт датасета в формат YOLO-segmentation, позволяющий сформировать полноценный обучающий датасет без ручной разметки.

5.1. Конвейер подготовки данных

Процесс подготовки включал следующие этапы:

- Извлечение сегментационных изображений из датасета MidAir — загрузка семантических карт. Segmentation в MidAir — это одноканальная карта классов: где каждый пиксель хранит номер класса. Так как классов небольшое количество, а именно в целевой траектории всего 6 классов, то изображение из segmentation выглядит как чёрный квадрат, но, если умножить все пиксели одного класса на 255, можно получить сегментированное отображение этого самого класса. Пример такого отображения класса деревьев представлено на изображении 1.

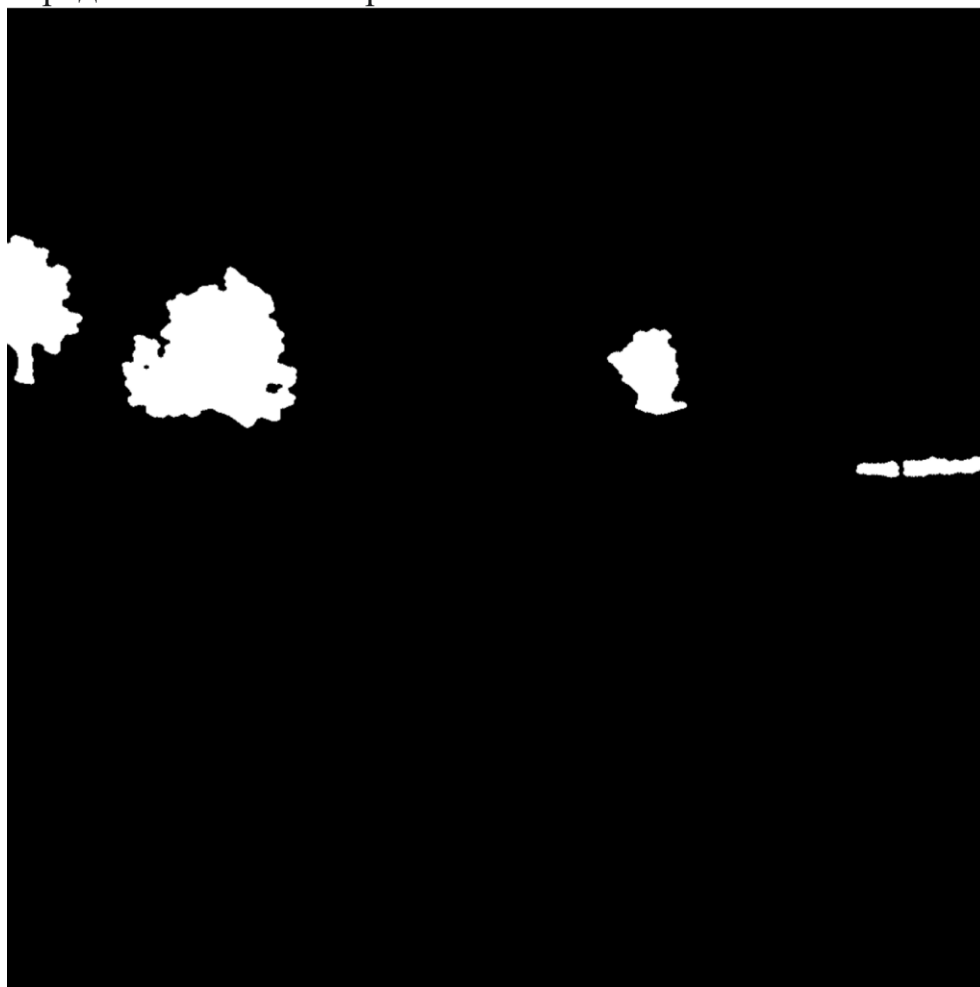


Рисунок 1 – Отображение класса деревьев

– Определение уникальных меток классов — идентификация номеров классов. С помощью скрипта из приложения А, было собрано по несколько примеров для каждого класса, чтобы сопоставить номер класса и его семантическое значение. По итогу был создан файл `label_map.json`, который связывает числовые метки датасета с семантическими классами. Потом были отобраны целевые классы, а именно `tree`, `rock`, `river`. Итоговый файл `label_map.json` представлен на рисунке 2.

```
{
  "2": "tree",
  "6": "rock",
  "8": "river"
}
```

Рисунок 2 – Файл `label_map.json`

Далее, с помощью скрипта из приложения Б было сделано следующее:

– Выделение связанных компонент (`connected components`) для каждого класса — для каждого целевого класса на каждом изображении применялся алгоритм разметки связанных областей, позволяющий разделить пиксели одного класса на отдельные экземпляры объектов.

– Фильтрация мелких объектов — исключение из разметки компонент, площадь которых ниже установленного порогового значения, во избежание шумовых меток, не несущих навигационной ценности.

– Генерация полигональных масок YOLO — для каждой отфильтрованной связанной компоненты извлекается контурный полигон, координаты вершин которого нормируются в диапазон `[0, 1]` и записываются в формате YOLO-seg (`class_id x1 y1 x2 y2 ... xN yN`).

По итогу работы скрипта из приложения Б был получен датасет, состоящий из RGB изображения и принадлежащему ему `label`, в котором хранятся координаты и полигоны объектов целевых классов на изображении. На рисунке 3 представлено RGB изображение, на котором цветами выделены полигоны классов, а также выделены квадратами области классов с подписями. [3]



Рисунок 3 – Объединение RGB и labels

5.2. Структура и разбиение датасета

Сформированный датасет был разделён на обучающую и валидационную выборки в соотношении 80/20, что является стандартной практикой при обучении нейросетевых моделей. Структура датасета соответствует требованиям фреймворка Ultralytics YOLO:

- images/train — RGB-изображения обучающей выборки;
- images/val — RGB-изображения валидационной выборки;
- labels/train — файлы разметки YOLO-seg для обучающей выборки;
- labels/val — файлы разметки YOLO-seg для валидационной выборки.

Каждому изображению соответствует текстовый файл разметки с аналогичным именем, содержащий описания всех аннотированных экземпляров объектов в нормированном формате YOLO. Балансировка классов в датасете обеспечивалась равномерным отбором изображений из различных сцен MidAir.

5.3 Виды сегментации изображений

Сегментация изображений — одна из ключевых задач компьютерного зрения, направленная на разбиение изображения на смысловые области. В данной работе используются и сравниваются два основных подхода к сегментации, принципиально отличающихся по формату выходных данных и применимости в навигационных задачах.

Semantic Segmentation (семантическая сегментация). Семантическая сегментация — задача попиксельной классификации, при которой каждому пикселю присваивается метка класса. Все объекты одного класса неразличимы: три отдельных дерева будут представлены единой слитой областью класса `tree`. Результат — плотная маска классов размером, совпадающим с входным изображением. Типичные архитектуры: FCN, DeepLab, U-Net, SegFormer.

Instance Segmentation (сегментация экземпляров). Сегментация экземпляров расширяет семантическую: каждый отдельный экземпляр объекта получает независимую маску и уникальный идентификатор. Три рядом стоящих дерева будут представлены тремя отдельными масками с собственными центроидами и ограничивающими прямоугольниками. Типичные архитектуры: Mask R-CNN, YOLACT, YOLO-seg.

Сравнение для задачи навигации БПЛА

Для навигации БПЛА по визуальным ориентирам `instance segmentation` имеет принципиальное преимущество: информационный вектор сцены строится из пространственных координат центроидов конкретных объектов, расстояний между ними и их взаимного расположения. Эти характеристики доступны только при разделении на экземпляры. Семантическая сегментация (SegFormer) включена в работу как сравнительная база — для оценки качества распознавания классов в парадигме, не требующей разделения на экземпляры.

6. Обучение моделей сегментации

6.1. Выбор архитектуры

Для решения задачи сегментации визуальных ориентиров была выбрана семейство архитектур YOLO (You Only Look Once) — одностадийных детекторов и сегментаторов, известных высокой скоростью работы при конкурентоспособной точности[7]. Данный выбор обусловлен требованиями бортовых систем реального времени: алгоритм навигации должен обрабатывать видеопоток с частотой не менее 25–30 кадров в секунду, что исключает применение вычислительно тяжёлых двухстадийных детекторов.

В исследовании рассматривались две генерации архитектуры в трёх размерных вариантах:

- YOLOv8 (версия 8) — актуальная на момент проведения исследований архитектура. [5]

- YOLOv11 (версия 11) — последующая итерация, предлагающая ряд архитектурных улучшений, направленных на повышение точности при сохранении скоростных характеристик. [6]

Каждая архитектура тестировалась в трёх вариантах масштабирования: nano (n) — минимальная модель с наименьшим числом параметров, ориентированная на максимальную производительность; small (s) — промежуточный вариант; medium (m) — модель увеличенной ёмкости для повышенной точности. [9]

6.2. Параметры обучения

Все шесть моделей обучались с идентичными гиперпараметрами для обеспечения корректности сравнения:

- размер входного изображения: 640×640 пикселей — стандартный размер для YOLO-моделей, обеспечивающий баланс между детальностью и скоростью обработки;

- размер мини-батча (batch size): 8 — выбран с учётом доступного объёма видеопамати GPU; [2]

- число эпох обучения: 40 — достаточное для сходимости метрик на данном объёме датасета;

- оптимизатор: SGD с моментом и стандартным расписанием снижения скорости обучения;

- аугментации: стандартный набор Ultralytics YOLO (мозаичное объединение, случайные кадрирования, изменения яркости/насыщенности).

Код обучения и вспомогательный файл dataset.yaml [8], в котором хранятся пути до файлов датасета и названия классов, представлен на рисунке 4.



Рисунок 4 – Код обучения и dataset.yaml

6.3 Архитектура SegFormer

SegFormer — современная архитектура семантической сегментации, предложенная Xie и соавторами (NVIDIA, 2021) [10]. Модель объединяет иерархический Transformer-энкодер с лёгким MLP-декодером, обеспечивая высокую точность при умеренных вычислительных затратах.

Encoder: Mix Transformer (MiT). Энкодер SegFormer основан на архитектуре Mix Transformer (MiT) — иерархическом трансформере, извлекающем многомасштабные признаки на четырёх уровнях разрешения. Ключевые особенности:

- Overlapping Patch Merging — перекрывающееся разбиение на патчи, сохраняющее локальную пространственную непрерывность;
- Efficient Self-Attention — снижение вычислительной сложности за счёт пространственного уменьшения последовательности ключей и значений с коэффициентами $R = \{64, 16, 4, 1\}$ для разных уровней;
- Mix-FFN — Feed-Forward Network со свёрточным слоем 3×3 вместо явного позиционного кодирования.

Decoder: All-MLP Head. Декодер SegFormer намеренно прост: четыре линейных слоя обрабатывают признаки четырёх уровней энкодера. Выходы апсэмплируются до разрешения $H/4 \times W/4$, конкатенируются и пропускаются через финальный слой классификации. Такая архитектура обеспечивает эффективное объединение локальных и глобальных признаков.

Варианты масштабирования

В данной работе исследованы три варианта SegFormer:

- SegFormer-B0: MiT-B0, ~3.7 млн параметров — минимальные вычислительные затраты;
- SegFormer-B1: MiT-B1, ~13.7 млн параметров — компромисс точности и скорости;
- SegFormer-B2: MiT-B2, ~27.4 млн параметров — повышенная точность.

6.4 Параметры обучения SegFormer

Обучение моделей SegFormer проводилось на том же датасете MidAir (4 класса: background, tree, rock, river) с параметрами, рекомендованными библиотекой HuggingFace Transformers:

- число эпох: 20;
- размер батча: 8;
- оптимизатор: AdamW с cosine scheduler;
- функция потерь: Cross-Entropy Loss;
- разрешение входа: 512×512 пикселей.

Код обучения представлен в приложении Г.

7. Результаты экспериментов

7.1. Метрики точности

По результатам обучения для каждой модели были вычислены следующие метрики на валидационной выборке: Precision (точность), Recall (полнота), mAP50 (средняя точность при IoU=0.5) и mAP50-95 (средняя точность в диапазоне IoU от 0.5 до 0.95 с шагом 0.05). Метрики вычислены отдельно для задачи ограничивающего прямоугольника (box) и задачи сегментации маски (mask). Результаты представлены в таблице 1.

Таблица 1 – Результаты обучения

Модель	P (box)	R (box)	mAP50 (box)	mAP50- 95 (box)	P (mask)	R (mask)	mAP50 (mask)	mAP50- 95 (mask)
YOLOv8n	0.952	0.953	0.987	0.884	0.957	0.948	0.982	0.753
YOLOv8s	0.961	0.954	0.989	0.907	0.971	0.948	0.986	0.784
YOLOv8m	0.972	0.955	0.990	0.918	0.972	0.952	0.986	0.792
YOLO11n	0.953	0.954	0.985	0.877	0.950	0.950	0.978	0.743
YOLO11s	0.961	0.962	0.989	0.905	0.962	0.962	0.985	0.780
YOLO11m	0.973	0.955	0.989	0.913	0.973	0.955	0.985	0.792

7.2 F1-критерий для YOLO-seg

F1-score (критерий Фишера) является гармоническим средним между Precision и Recall и позволяет оценить баланс между полнотой обнаружения ориентиров и точностью их локализации. Для задачи сегментации маски F1 вычисляется по формуле:

$$F1 = 2 \cdot \text{Precision}(\text{mask}) \cdot \text{Recall}(\text{mask}) / (\text{Precision}(\text{mask}) + \text{Recall}(\text{mask}))$$

Ключевым инструментом для анализа является кривая F1–Confidence: зависимость F1-score от порога уверенности детектора. Оптимальный порог выбирается в точке максимума F1-кривой. Кривые представлены ниже.

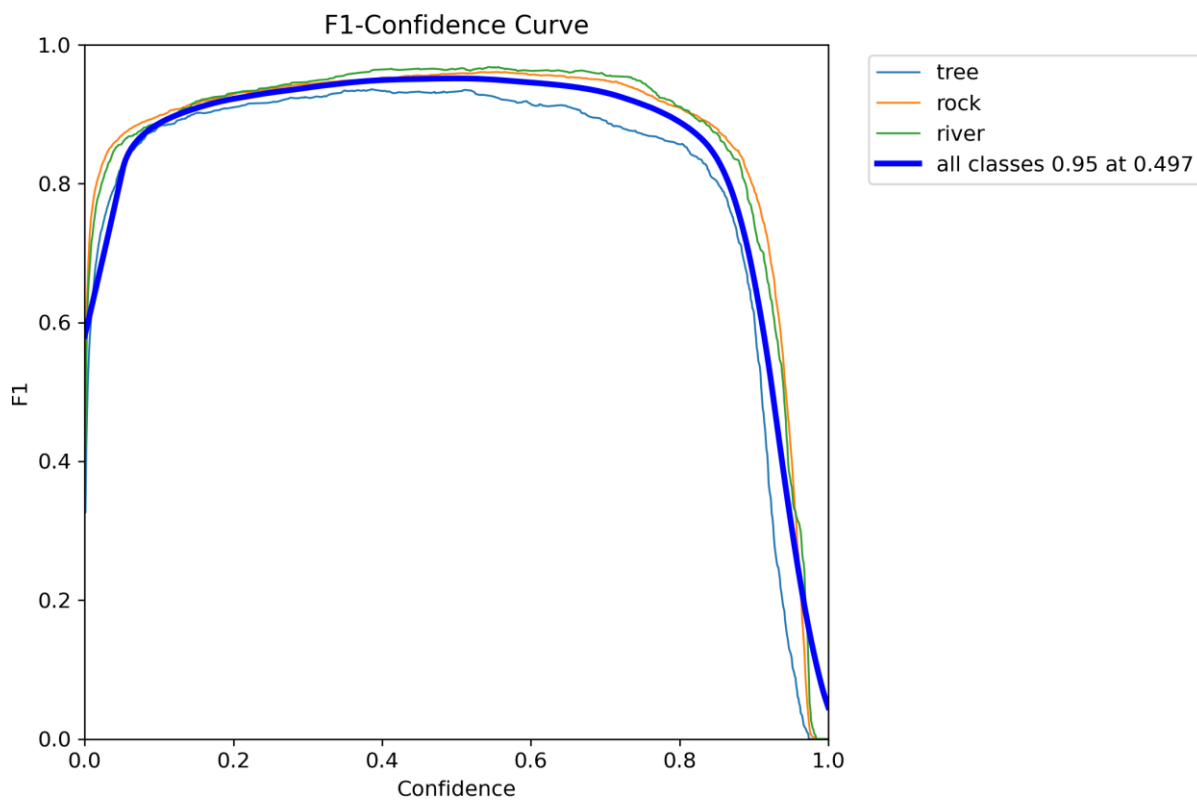


Рисунок 5 — F1-кривая модели YOLOv8n-seg

MaskF1-Confidence Curves — сравнение всех моделей YOLO-seg

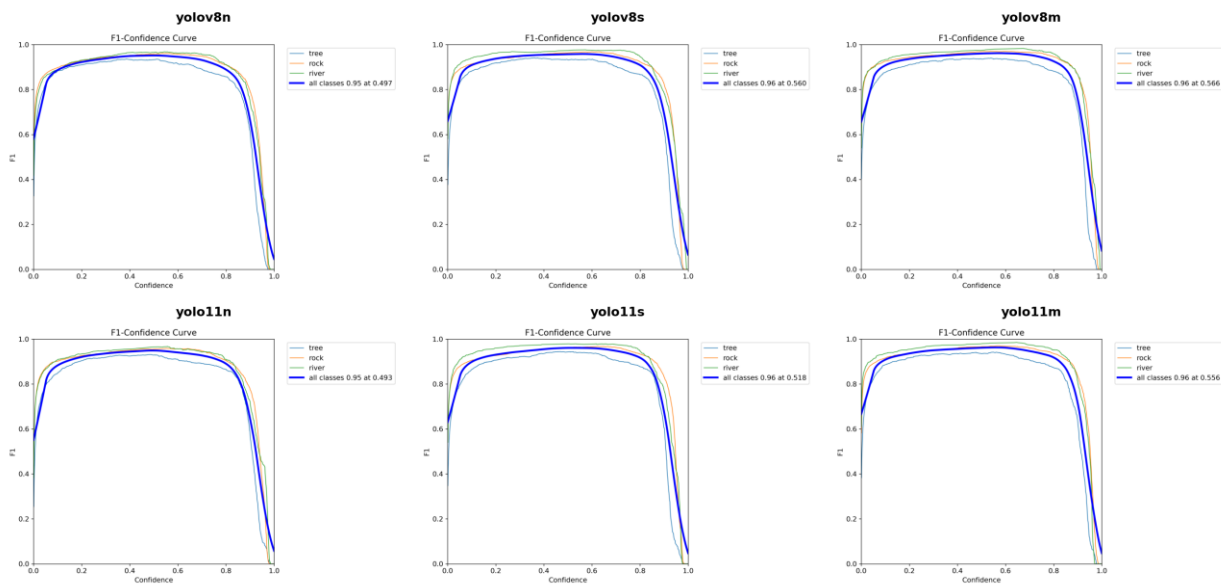


Рисунок 6 — Сравнение F1-кривых моделей YOLO-seg

Таблица 2 — F1-score моделей YOLO-seg (mask) при оптимальном пороге уверенности

Модель	Precision (mask)	Recall (mask)	F1-score (mask)	mAP50 (mask)	mAP5 0-95 (mask)
YOLOv8					
YOLOv8n	0.957	0.948	0.9525	0.982	0.753
YOLOv8s	0.971	0.948	0.9594	0.986	0.784
YOLOv8m	0.972	0.952	0.9619	0.986	0.792
YOLO11					
YOLO11n	0.950	0.950	0.9500	0.978	0.743
YOLO11s	0.962	0.962	0.9620	0.985	0.780
YOLO11m	0.973	0.955	0.9639	0.985	0.792

* F1 вычислен как $2 \cdot P \cdot R / (P + R)$ по значениям Precision и Recall на валидационной выборке при $\text{conf}=0.5$.

Все модели демонстрируют высокий F1-score в диапазоне 0.952–0.964. Наилучший F1 показывает YOLO11s (0.962), наименьший — YOLOv8n (0.952). Разрыв невелик, что подтверждает общую состоятельность всех обученных моделей.

7.3 Производительность моделей YOLO (GPU)

Наряду с метриками точности для каждой модели была проведена бенчмарковая оценка скорости работы на GPU. Измерялось среднее время инференса на одно изображение (мс) по 100 запускам. Результаты представлены в таблице 3.

Таблица 3 — Среднее время инференса YOLO-seg на GPU

Модель	Среднее время инференса (мс)	FPS
YOLOv8		
YOLOv8n	28.94	34.6
YOLOv8s	29.02	34.5
YOLOv8m	32.74	30.5
YOLO11		
YOLO11n	31.29	32.0
YOLO11s	32.38	30.9
YOLO11m	35.63	28.1

Скрипт бенчмарка приведён в приложении В.

7.4 Визуализация результатов сегментации YOLO

Ниже представлены места для примеров работы моделей YOLO-seg на изображениях валидационной выборки датасета MidAir. На каждом изображении отображаются индивидуальные маски для каждого обнаруженного экземпляра объекта с меткой класса и оценкой уверенности.

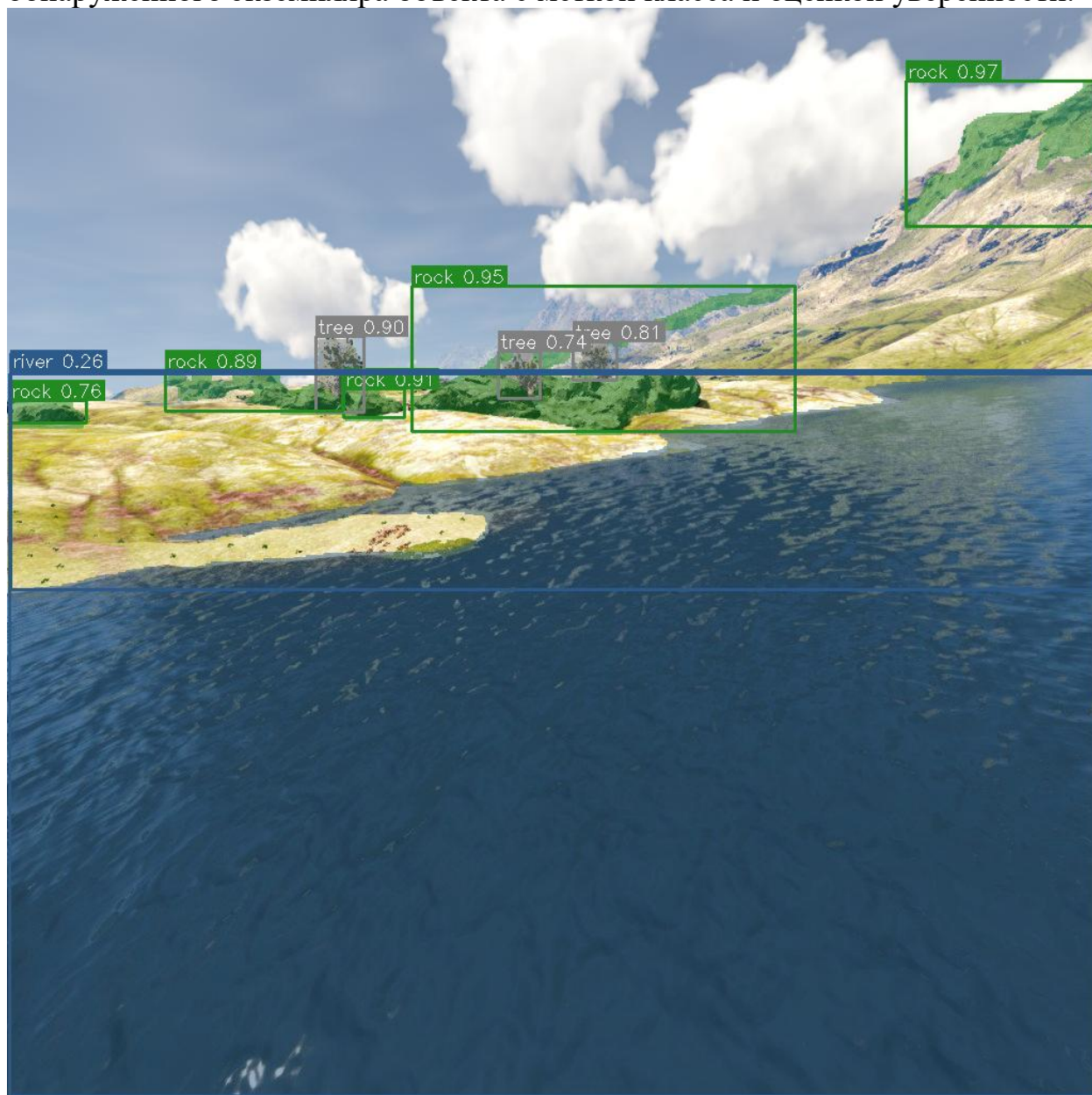


Рисунок 7 — Instance segmentation YOLOv8n-seg: зелёный — tree, коричневый — rock, синий — river

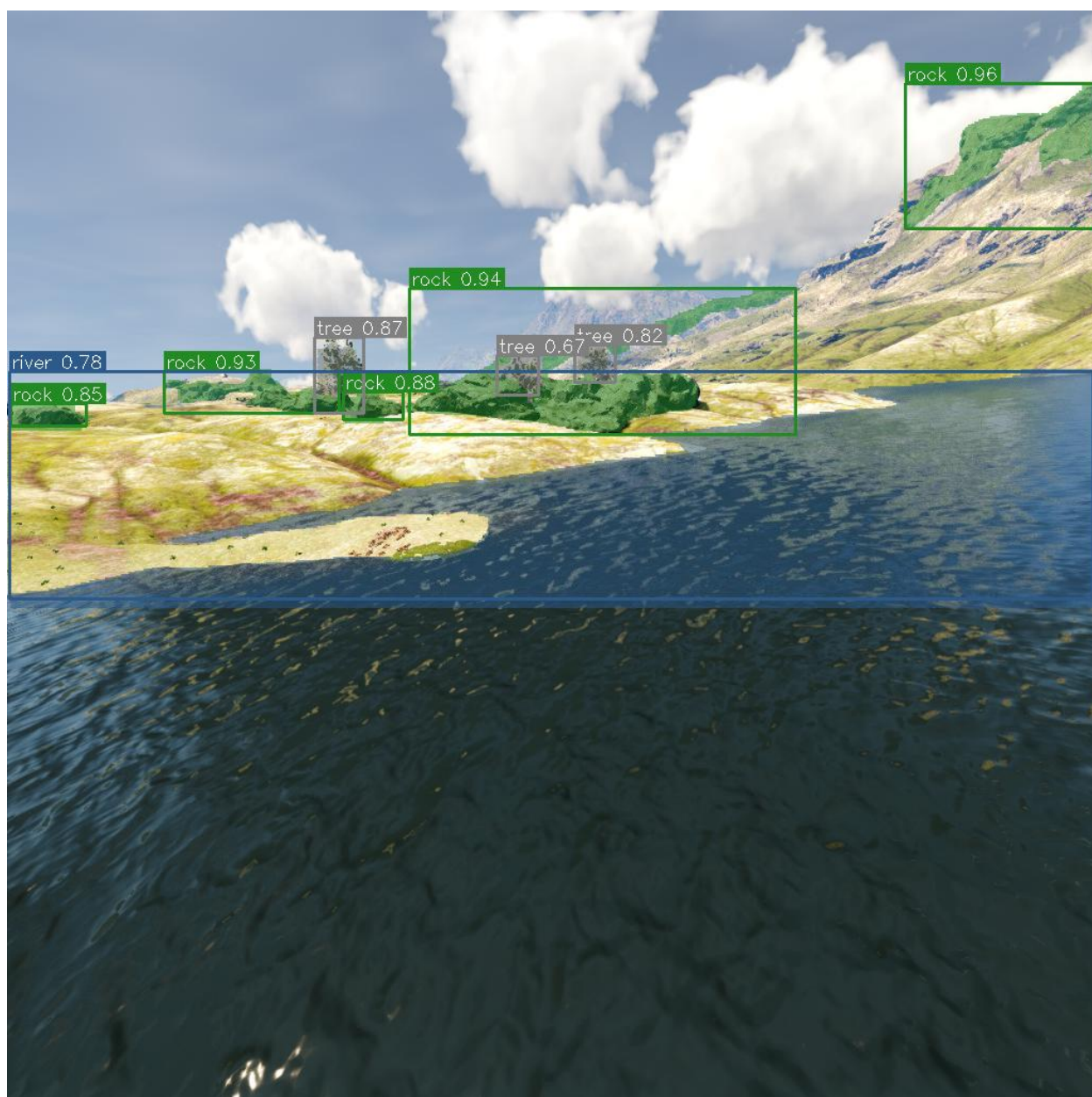


Рисунок 8 — Instance segmentation YOLOv8s-seg

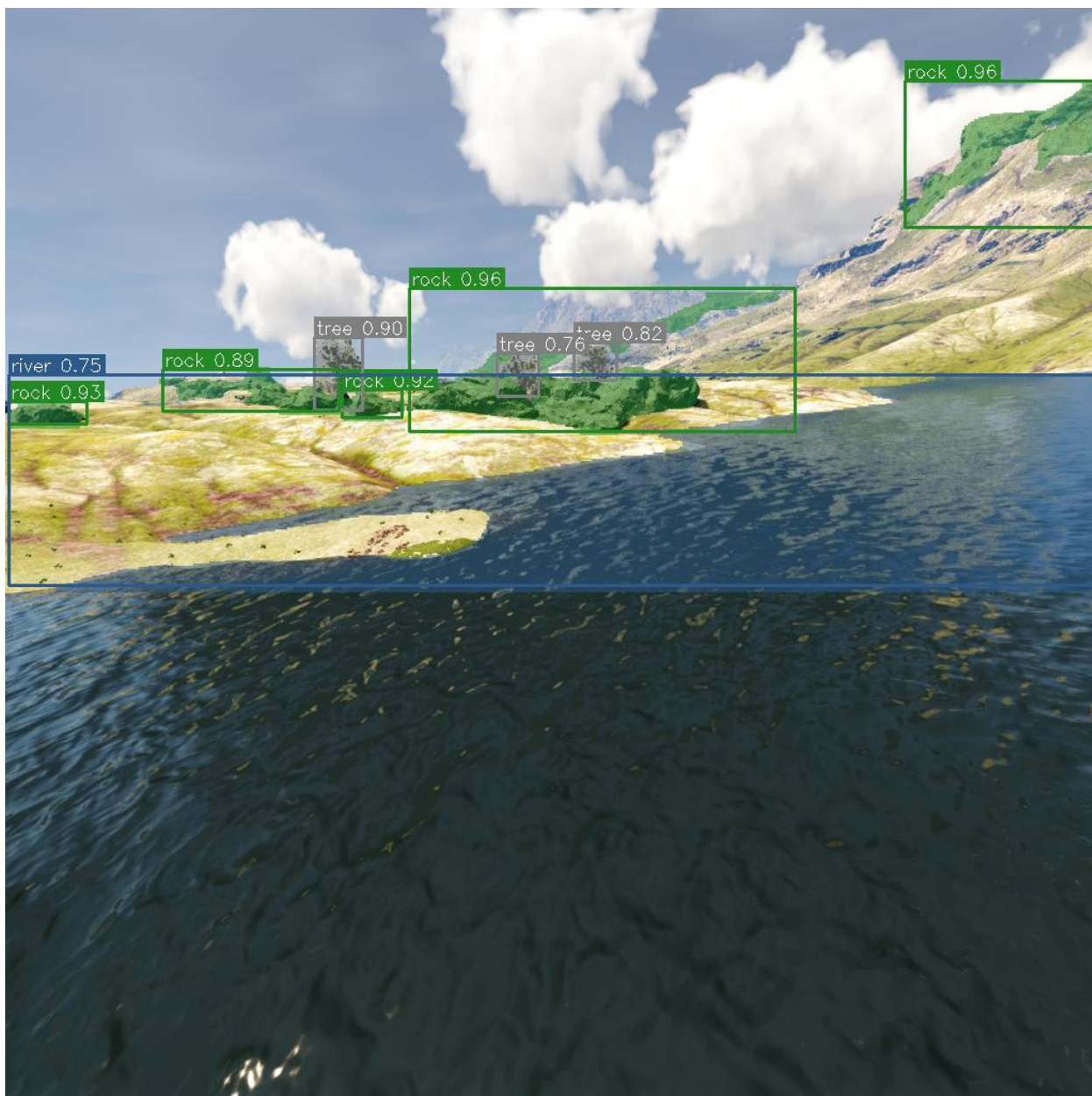


Рисунок 9 — Instance segmentation YOLOv8m-seg

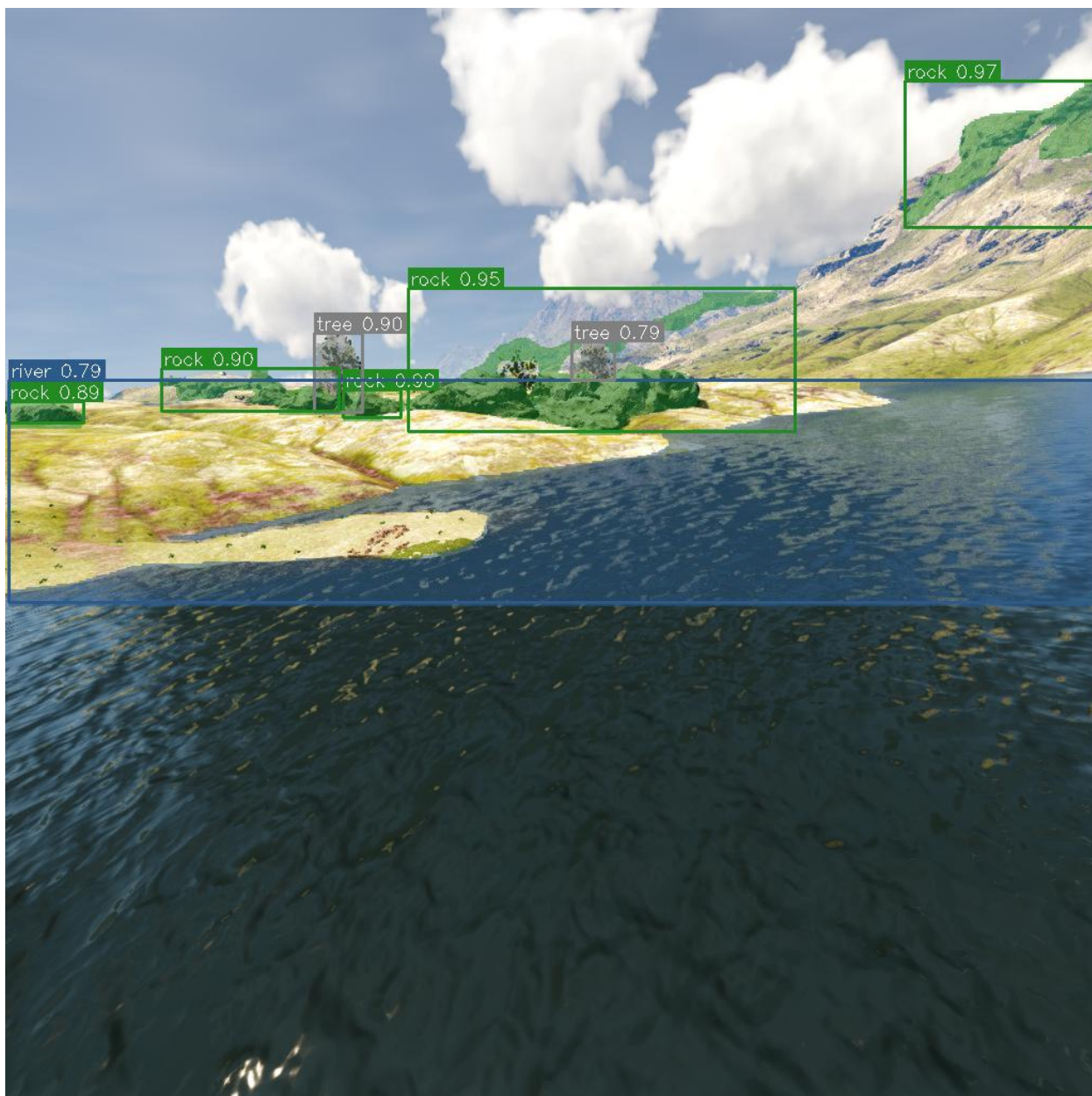


Рисунок 10 — Instance segmentation YOLO11n-seg

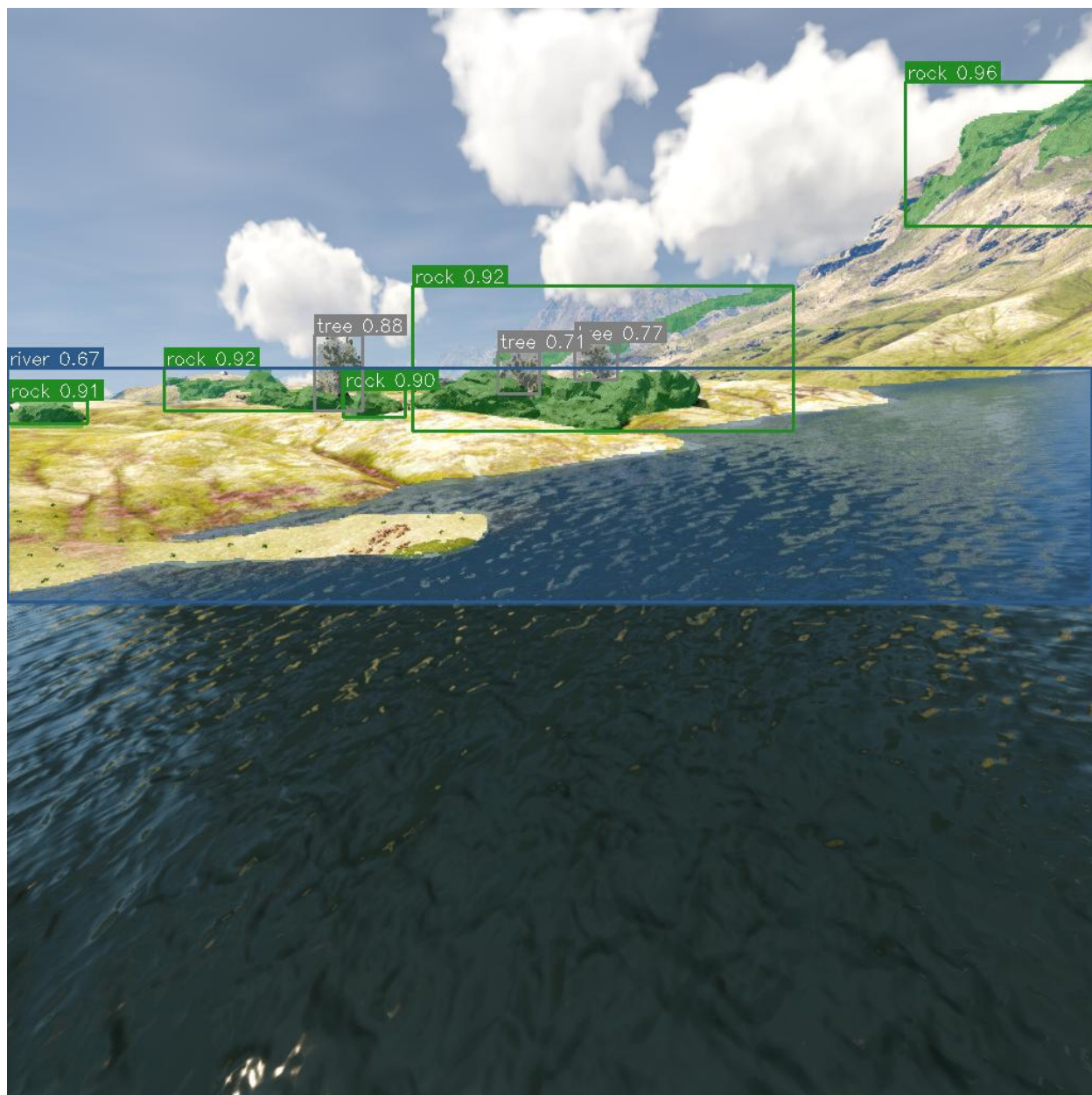


Рисунок 11 — Instance segmentation YOLO11s-seg

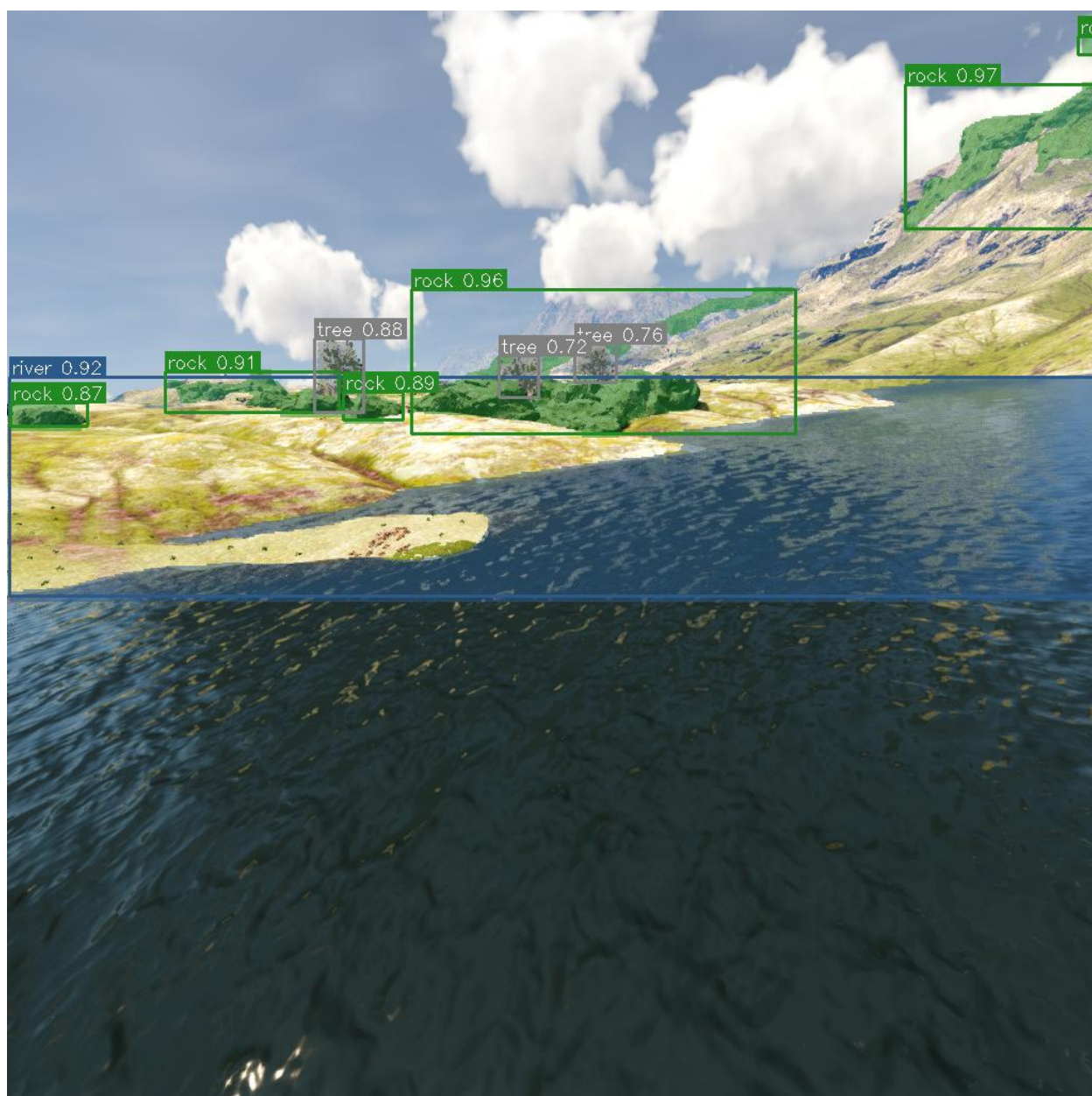


Рисунок 12 — Instance segmentation YOLO11m-seg

7.5 Результаты обучения SegFormer

Все три варианта SegFormer (B0, B1, B2) обучались в течение 20 эпох на том же датасете MidAir. Для оценки использовались стандартные метрики semantic segmentation: Eval Loss, mean IoU (mIoU), Overall Accuracy и per-class IoU. Результаты представлены в таблице 4.

Таблица 4 — Метрики качества моделей SegFormer на валидационной выборке MidAir

Модель	Eval Loss	mean IoU	IoU bg	IoU tree	IoU rock	IoU river
SegFormer-B0	0.0217	0.908	0.989	0.964	0.687	0.992
SegFormer-B1	0.0196	0.911	0.990	0.968	0.693	0.993
SegFormer-B2	0.0185	0.913	0.990	0.971	0.696	0.993

Все три модели SegFormer показывают высокое overall accuracy (>0.99) и mean IoU (>0.907). Наиболее сложным классом является rock: $\text{IoU} \approx 0.687\text{--}0.696$ — заметно ниже остальных классов, что объясняется высокой вариативностью внешнего вида скальных образований. Аналогичная тенденция наблюдается и у YOLO-seg.



Рисунок 13 — Semantic segmentation SegFormer-B0. Слева — оригинал, в центре — маска классов, справа — overlay



Рисунок 14 — Semantic segmentation SegFormer-B1



Рисунок 15 — Semantic segmentation SegFormer-B2

7.6 Бенчмарк производительности на Raspberry Pi 5

Для оценки применимости в бортовых условиях проведено тестирование скорости инференса на одноплатном компьютере Raspberry Pi 5 (4-ядерный Cortex-A76, 8 ГБ RAM). Платформа не имеет аппаратного NPU, что характерно для бюджетных бортовых компьютеров БПЛА. Тестирование проводилось при разрешении 640×640.

Таблица 5 — Производительность моделей на Raspberry Pi 5 (640×640, формат PyTorch)

Модель	Формат	Время инференса (мс)	FPS
YOLO-seg (.pt, PyTorch CPU)			
YOLOv8n	PyTorch .pt	686.22	1.5
YOLOv8s	PyTorch .pt	1905.25	0.5
YOLOv8m	PyTorch .pt	4569.23	0.2
YOLO11n	PyTorch .pt	689.87	1.4
YOLO11s	PyTorch .pt	1801.49	0.6
YOLO11m	PyTorch .pt	5658.22	0.2
SegFormer (PyTorch CPU)			
SegFormer-B0	PyTorch CPU	1650.45	0.6
SegFormer-B1	PyTorch CPU	2331.46	0.4
SegFormer-B2	PyTorch CPU	6751.97	0.1

Скрипт бенчмарка приведён в приложении В.

8. Анализ и сравнение моделей

8.1. Анализ метрик точности

Анализ результатов, приведённых в таблице 1, позволяет сделать ряд принципиальных наблюдений. Все шесть обученных моделей демонстрируют весьма высокие значения mAP50 как для детекции (0.985–0.990), так и для сегментации (0.978–0.986), что свидетельствует о качественной постановке задачи, достаточном объёме и корректной разметке обучающих данных, а также об адекватности выбранных классов ориентиров для визуальной навигации.

Показатель mAP50-95 — более строгая метрика, учитывающая качество масок при высоких порогах совпадения — для задачи сегментации находится в диапазоне 0.743–0.792, что является закономерным снижением по сравнению с mAP50. Разброс значений mAP50-95 по моделям более информативен для ранжирования, чем mAP50.

Среди моделей YOLOv8 наиболее высокую точность демонстрирует YOLOv8m: mAP50-95 (box) = 0.918, mAP50-95 (mask) = 0.792. Модели YOLOv8s и YOLOv8n уступают ей в точности, особенно по строгой метрике mAP50-95. Аналогичная закономерность прослеживается в семействе YOLOv11: YOLOv11m (mAP50-95 mask = 0.792) незначительно превосходит YOLOv11s (0.780) и существенно — YOLOv11n (0.743). Примечательно, что YOLOv11s демонстрирует более высокий Recall (box и mask $\approx$ 0.962) по сравнению со всеми остальными моделями, что может быть предпочтительным в сценариях навигации, где пропуск ориентира критичнее ложной тревоги.

При сравнении архитектур YOLOv8 и YOLOv11 одного размера можно отметить, что YOLOv11m незначительно уступает YOLOv8m по mAP50-95 (mask): 0.792 против 0.792 (практически идентичны), однако YOLOv11n заметно проигрывает YOLOv8n: 0.743 против 0.753. В то же время YOLOv11s практически сравнялась с YOLOv8s по всем метрикам.

8.2. Анализ производительности

Данные таблицы 2 демонстрируют закономерную зависимость времени инференса от размера модели: nano-варианты работают быстрее small, small — быстрее medium. Модель YOLOv8n обеспечивает наибольшую производительность: 34.6 FPS (28.94 мс), что близко к YOLOv8s (34.5 FPS, 29.02 мс). Обе YOLOv8 nano и small превышают порог 30 FPS, необходимый для навигации в реальном времени.

Модели семейства YOLOv11 несколько уступают аналогам YOLOv8 по скорости: YOLOv11n выдаёт 32.0 FPS (31.29 мс), YOLOv11s — 30.9 FPS (32.38 мс). Самой медленной является YOLOv11m: 28.1 FPS (35.63 мс) — единственная модель, не преодолевающая порог 30 FPS.

Следует учитывать, что приведённые показатели производительности получены на конкретном аппаратном обеспечении. На бортовом компьютере

БПЛА с меньшей вычислительной мощностью (например, Raspberry PI5) все значения FPS снизятся, что делает выбор лёгкой и быстрой модели ещё более критичным.

8.3. Выбор оптимальной модели

При выборе модели для интеграции в систему навигации БПЛА необходимо соблюдать баланс между точностью сегментации (определяет надёжность обнаружения ориентиров) и производительностью (определяет частоту обновления навигационных данных). Критически важно, чтобы FPS системы не опускался ниже 25–30 кадров в секунду для обеспечения стабильной работы в реальном времени.

С учётом данных требований модель YOLOv8s представляется наиболее сбалансированным выбором: она обеспечивает $mAP_{50-95}(\text{mask}) = 0.784$ при производительности 34.5 FPS. Незначительное превосходство YOLOv8m в точности ($mAP_{50-95}(\text{mask}) = 0.792$, разница всего 0.8%) достигается ценой снижения FPS с 34.5 до 30.5 — граничного значения для систем реального времени. Для сценариев, где точность критична, а вычислительные ресурсы достаточны, предпочтение следует отдать YOLOv8m.

Модели YOLOv11s и YOLOv11m демонстрируют сопоставимую с YOLOv8 точность, однако уступают в скорости, что снижает их привлекательность для бортового применения. Nano-варианты обеих архитектур, несмотря на максимальную скорость, показывают заметно меньшую точность по строгим метрикам, что может привести к частым пропускам ориентиров в сложных условиях наблюдения.

8.4 Сравнение YOLO-seg и SegFormer

Прямое числовое сравнение YOLO-seg и SegFormer затруднено ввиду различия решаемых задач. YOLO-seg оценивается метриками instance segmentation — mAP_{50-95} по маскам экземпляров. SegFormer оценивается метриками semantic segmentation — mean IoU и Overall Accuracy. Числовое сравнение носит ориентировочный характер.

Таблица 6 — Сводное сравнение всех моделей по точности и производительности

Модель	Тип задачи	mAP50-95 / mIoU	F1-score (mask/—)	FPS GPU	FPS RPi5
Instance Segmentation — YOLO-seg					
YOLOv8n	Instance Seg	0.753	0.9525	34.6	1.5
YOLOv8s	Instance Seg	0.784	0.9594	34.5	0.5
YOLOv8m	Instance Seg	0.792	0.9619	30.5	0.2
YOLO11n	Instance Seg	0.743	0.9500	32.0	1.4
YOLO11s	Instance Seg	0.780	0.9620	30.9	0.6
YOLO11m	Instance Seg	0.792	0.9639	28.1	0.2
Semantic Segmentation — SegFormer					
B0 SegFormer-	Semantic Seg	0.908	—	~40	0.6
B1 SegFormer-	Semantic Seg	0.911	—	~30	0.4
B2 SegFormer-	Semantic Seg	0.913	—	~22	0.1

* Для YOLO-seg: основная метрика — mAP50-95 (mask); F1 при conf=0.5. Для SegFormer: mIoU — mean Intersection over Union; F1 не применяется в стандартной семантической постановке.

SegFormer демонстрирует более высокие значения mIoU (0.908–0.913) по сравнению с mAP50-95 mask у YOLO-seg (0.743–0.792). Однако это сравнение некорректно трактовать как превосходство SegFormer: mIoU для семантической сегментации и mAP50-95 для instance segmentation измеряют разные вещи. Принципиальное преимущество YOLO-seg для навигации: система сопоставления сцены требует координат центроидов отдельных объектов, которые SegFormer принципиально не предоставляет.

8.5 Оптимизация модели: ONNX и NCNN

ONNX — Open Neural Network Exchange

ONNX (Open Neural Network Exchange) — открытый формат представления моделей машинного обучения, разработанный Microsoft и Facebook (Meta) в 2017 году. Описывает вычислительный граф модели в виде стандартизированных операторов в бинарном формате Protocol Buffers, независимом от конкретного фреймворка.

ONNX Runtime реализует ключевые оптимизации: Graph Optimizations (свёртка константных узлов, operator fusion), Memory Planning и подключаемые Execution Providers. На ARM-платформах

ACLExecutionProvider обеспечивает использование SIMD-инструкций, давая ускорение в 3–5 раз относительно PyTorch CPU.

Экспорт YOLOv8n-seg в ONNX:

```
yolo export model=best.pt format=onnx imgsz=640
```

NCNN — Neural network Computing for Neural Networks

NCNN — высокопроизводительный фреймворк для инференса нейронных сетей, разработанный Tencent и оптимизированный специально для мобильных платформ и встраиваемых систем. Написан на чистом C++11 без внешних зависимостей.

Ключевые особенности NCNN:

- Оптимизация под ARM NEON SIMD — векторные инструкции для параллельных вычислений;
- Поддержка Vulkan GPU — задействует встроенный GPU через Vulkan API;
- Квантизация INT8 и FP16 — снижение объёма вычислений без существенной потери точности;
- Минимальный overhead — отсутствие Python-рантайма и JIT-компиляции.

Экспорт YOLOv8n-seg в NCNN:

```
yolo export model=best.pt format=ncnn imgsz=640
```

Результаты оптимизации

В таблице 7 приведены ориентировочные значения, полученные на модели YOLOv8n-detect. Значения для seg-версии подлежат уточнению после экспорта (ожидается незначительное снижение ввиду дополнительной маски-головы).

Таблица 7 — Производительность YOLOv8n-seg в различных форматах на Raspberry Pi 5

Модель	Формат	Время инференса (мс)	FPS
YOLOv8n-seg на Raspberry Pi 5 (ориентировочные данные)*			
YOLOv8n-seg	PyTorch .pt (baseline)	686.22	1.5
YOLOv8n-seg	ONNX Runtime (CPU)	~180	~5.6
YOLOv8n-seg	NCNN (640×640)	~136	~7.4

Применение NCNN позволяет достичь ~7.4 FPS при разрешении 640×640 — прирост в 4.9 раза. Снижение разрешения до 320×320 дополнительно повысит результат в дальнейшем. При добавлении модуля аппаратного ускорения Raspberry Pi AI HAT+ (26 TOPS) ожидается 15–30 FPS для YOLOv8n-seg.

9 Анализ достаточности частоты кадров для навигации по визуальным ориентирам

Одним из ключевых практических вопросов при развёртывании системы визуальной навигации на борту одноплатного компьютера является вопрос о минимальной достаточной частоте кадров (FPS) нейросетевой сегментации. В разделе 7.6 показано, что при формате NCNN и разрешении 640×640 модель YOLOv8n-seg достигает $\sim 7,4$ FPS, а при снижении разрешения до 320×320 ожидается достижение 13–14 FPS. Настоящий раздел обосновывает, что данная частота кадров является достаточной для задачи локализации БПЛА по природным ориентирам при скоростях полёта до 20 м/с.

9.1 Специфика задачи. Сегментация ориентиров против высокоскоростного слежения

Принципиально важно различать два класса задач компьютерного зрения, для которых нормы по FPS существенно различаются:

Высокоскоростное слежение (High-Speed Tracking) — отслеживание малых быстрых объектов (других БПЛА, транспортных средств), где каждый кадр критически важен. Для таких задач традиционно предъявляется требование $\geq 25\text{--}30$ FPS [12].

Навигация по статичным ориентирам (Landmark-Based Navigation) — обнаружение и описание крупных неподвижных объектов местности (деревья, скальные образования, водные объекты) для определения местоположения аппарата. В данном случае ориентиры сохраняют своё местоположение и визуальные характеристики между кадрами, а пропущенные кадры компенсируются данными IMU. Именно эта задача решается в настоящей работе.

9.2 Геометрический анализ межкадрового смещения

Для оценки допустимости 13–14 FPS при скорости БПЛА до 20 м/с проведём расчёт межкадрового смещения ориентира в пикселях.

Исходные данные:

- Скорость БПЛА: $V = 20$ м/с
- Частота кадров: $F = 14$ FPS $\rightarrow$ интервал $\Delta t = 1/14 \approx 0,071$ с
- Горизонтальное линейное смещение за один кадр: $\Delta X = V \cdot \Delta t = 20 \cdot 0,071 \approx 1,43$ м
- Типовая высота полёта: $H = 50\text{--}100$ м
- Разрешение входного изображения: 320×320 пикселей
- Горизонтальный угол обзора камеры (FOV): $\sim 90^\circ$

Расчёт масштаба изображения (при $H = 50$ м, $\text{FOV} = 90^\circ$):

Горизонтальный охват сцены $= 2 \cdot H \cdot \tan(45^\circ) = 2 \cdot 50 \cdot 1 = 100$ м

Масштаб: 100 м / 320 пикселей $\approx 0,31$ м/пиксель

Межкадровое смещение ориентира в пикселях:

$$\Delta X_{px} = 1,43 \text{ м} / 0,31 \text{ м} \cdot \text{пкс}^{-1} \approx 4,6 \text{ пикселя}$$

Для высоты $H = 100 \text{ м}$ масштаб удваивается ($\approx 0,625 \text{ м/пиксель}$), смещение снижается до $\sim 2,3$ пикселя.

Таким образом, за один кадр при 14 FPS крупный природный ориентир (дерево, скальное образование), занимающий от нескольких десятков до нескольких сотен пикселей, сместится лишь на 2–5 пикселей. Это смещение абсолютно незначительно для задачи поддержания покрытия маской и отслеживания центроида объекта.

Данный расчёт согласуется с выводами Ghanizadegan & Hashim (2025) [13], исследовавших визуально-инерциальную навигацию БПЛА с 6 DoF: авторы показывают, что при работе с геодезически стабильными ориентирами IMU обеспечивает достаточную точность пропагации состояния между редкими обновлениями от визуального модуля.

9.3 Роль IMU в компенсации низкой частоты кадров

Фундаментальным свойством предлагаемой архитектуры является асинхронная работа двух каналов: визуального сегментатора (13–14 FPS) и инерциального измерительного блока IMU (типично 100–200 Гц). Данный подход — использование высокочастотного IMU для интерполяции положения между редкими кадрами визуальной обработки — хорошо изучен в литературе по визуально-инерциальной одометрии (VIO).

Как отмечают авторы обзора по V-SLAM [14], интегрирование IMU снимает пространственное ограничение на частоту визуальной системы и допускает низкочастотную обработку кадров при сохранении непрерывной оценки состояния. Аналогичный принцип реализован в системе Lee et al. (2022) [15], где EKF-фильтр объединяет ориентирную локализацию (редкие обновления от камеры) и данные IMU (высокочастотное распространение траектории) для БПЛА в условиях отсутствия GPS; авторы успешно применяли данный подход на реальных полётах.

Практическое подтверждение приведено в работе [16]: система визуальной одометрии на основе IMU-ассистированного KLT-трекера достигла стабильной навигации при частоте обработки изображений 5 FPS на бортовой платформе — значительно ниже рассматриваемого порога. Авторы явно указывают, что IMU «снимает пространственное ограничение, допуская низкочастотную обработку».

9.4 Сравнение с аналогичными задачами в литературе

Ряд опубликованных систем навигации и мониторинга БПЛА, решающих задачи, близкие к настоящей работе, демонстрирует успешную работу при значительно более низком FPS, чем традиционные 25–30:

Таблица 8 – Сравнение с аналогичными задачами

Система	Задача	FPS	Платформа
Визуальная одометрия с IMU [16]	Навигация, оценка эго-движения	5 FPS	Бортовой одноплатный компьютер
YOLO + DeepSORT, навигация по ЛЭП [17]	Обнаружение ориентиров (опор ЛЭП), локализация БПЛА	~15–20 FPS	GPU-платформа
YOLOv8 на Raspberry Pi 5 (SKD) [18]	Обнаружение БПЛА	~10 FPS	Raspberry Pi 5
YOLO-навигация в помещении (YOLOv4-tiny) [19]	Обход препятствий, следование маршруту	23 FPS	Jetson Nano
Робот UAV с отслеживанием цели [20]	Визуальное слежение, навигация	8–18 FPS	Бортовой

Примечательно, что задача из работы [20] — отслеживание быстро движущейся цели — является значительно более требовательной к частоте кадров, чем сегментация статичных природных ориентиров. Тем не менее авторы показывают успешную работу при среднем FPS = 18. Авторы прямо указывают: при скорости цели 5 м/с и частоте 5 FPS против 20 FPS цель перемещается на 1 м против 0,25 м между кадрами, что влечёт неопределённость в оценке состояния — однако для неподвижных ориентиров местности данная проблема не возникает в принципе, так как ориентиры не движутся.

9.5 Оценка достаточности 13–14 FPS

На основании проведённого анализа сформулированы следующие аргументы в пользу достаточности 13–14 FPS для задачи навигации по природным ориентирам:

1. Геометрический аргумент. Межкадровое смещение крупного природного ориентира при скорости 20 м/с и высоте 50–100 м составляет 2–5 пикселей при разрешении 320×320, что не нарушает перекрытия масок между кадрами.

2. Аргумент от природы ориентиров. Деревья, скальные образования и водные объекты являются статичными элементами ландшафта. В отличие от задачи отслеживания подвижной цели, ориентир присутствует в нескольких последовательных кадрах и не требует прогнозирования его местоположения с высокой временной точностью.

3. Аргумент IMU-компенсации. IMU работает на частоте 100–200 Гц и обеспечивает непрерывную оценку положения и ориентации между кадрами сегментации. Алгоритм EKF использует данные IMU для предсказания состояния БПЛА в промежутках между обновлениями визуального модуля.

Эффективность подобной схемы подтверждена в многочисленных публикациях [13, 14, 15, 16].

4. Аргумент от угловой динамики. При высотах полёта ≥ 50 м угловая скорость изменения сцены относительно мала: даже при 20 м/с полного поворота сцены не происходит за 1–2 межкадровых интервала (0,07–0,14 с). Система навигации оперирует не пиксельными координатами мгновенного кадра, а усреднёнными характеристиками сцены, что дополнительно снижает чувствительность к временным пропускам.

5. Прецедент в литературе. Работа [16] демонстрирует работоспособность визуально-инерциальной навигации при 5 FPS; работа [18] показывает, что 10 FPS на Raspberry Pi 5 признаётся авторами пригодным для «почти реального времени» при обнаружении БПЛА — задачи, сопоставимой по сложности.

Таким образом, частота 13–14 FPS при разрешении 320×320 с использованием модели YOLOv8n-seg в формате NCNN является практически достаточной для решения задачи сегментации природных ориентиров и построения информационного вектора сцены в системе навигации БПЛА, при условии её интеграции с IMU-фильтром. В дальнейшем применение аппаратного ускорителя Raspberry Pi AI HAT+ (26 TOPS) позволит достичь целевого значения 15–30 FPS при полном разрешении 640×640 .

10 Перспективы развития проекта

Данная работа закладывает экспериментальную основу для создания полноценной системы автономной навигации БПЛА. Следующие направления развития предусматриваются в рамках дальнейшего исследования:

- Реализация модуля сопоставления сцены с картой ориентиров — построение алгоритма локализации БПЛА на основе информационных векторов, формируемых из выходов YOLO-seg (центроиды, площади масок, взаимное расположение объектов).
- Разработка алгоритма построения и обновления карты ориентиров (landmark map) — запись маршрута в «обучающем» режиме с последующим использованием для навигации без GPS (SLAM-подобный подход).
- Интеграция с фильтром Калмана (EKF) — объединение данных визуальной локализации и IMU для непрерывной оценки положения и скорости аппарата между кадрами сегментации.
- Тестирование в симуляторе AirSim — развёртывание полного навигационного пайплайна в симуляторе с целью оценки точности удержания курса и скорости возврата при отклонении.
- Оптимизация модели под аппаратные ускорители — тестирование YOLOv8n-seg с Raspberry Pi AI HAT+ (26 TOPS) и NVIDIA Jetson Orin Nano для оценки достижимого FPS в реальных условиях.
- Расширение классов ориентиров и тестирование на реальных аэрофотоснимках — включение дополнительных устойчивых объектов (здания, дороги, пересечения) и адаптация к реальным условиям съёмки.

11 Выводы и заключение

В данной работе была решена задача сравнительного исследования методов сегментации изображений применительно к задаче обнаружения визуальных ориентиров для автономной навигации БПЛА в условиях потери GPS-сигнала.

Разработан автоматизированный конвейер подготовки обучающего датасета на основе семантических карт MidAir. Обучены и оценены девять моделей двух классов: шесть моделей instance segmentation семейств YOLOv8 и YOLO11 (nano, small, medium) и три модели semantic segmentation SegFormer (B0, B1, B2). Проведено тестирование производительности всех моделей на GPU и на одноплатном компьютере Raspberry Pi 5 с исследованием оптимизированных форматов ONNX и NCNN.

По результатам исследования сформулированы следующие ключевые выводы:

1. Instance segmentation (YOLO-seg) является принципиально более подходящей парадигмой для задачи навигации по визуальным ориентирам по сравнению с semantic segmentation (SegFormer): только instance seg предоставляет пространственные координаты центроидов отдельных объектов, необходимые для построения информационного вектора сцены и сопоставления с картой ориентиров.

2. Все шесть моделей YOLO-seg показали высокое качество сегментации ($mAP50 \text{ mask} > 0.978$, $F1 > 0.952$), подтверждая применимость подхода к задаче обнаружения навигационных ориентиров.

3. По совокупности критериев (FPS, $mAP50$, $F1$, потребление памяти, потенциал оптимизации) для интеграции в навигационную систему выбрана модель YOLOv8n-seg: 34.6 FPS на GPU, $mAP50 \text{ mask} = 0.982$, $F1 = 0.952$.

4. SegFormer-B0/B1/B2 показывают высокие значения $mIoU$ (0.908–0.913) при попиксельной классификации, однако не обеспечивают разделение на экземпляры и значительно медленнее на RPi5 (0.1–0.6 FPS).

5. Применение оптимизированных форматов ONNX и NCNN позволяет повысить производительность YOLOv8n-seg на Raspberry Pi 5 приблизительно в 4–5 раз (до ~ 7 FPS при 640×640), что при снижении разрешения и использовании аппаратного ускорителя открывает путь к практическому развёртыванию.

6. Наиболее проблемным для обоих классов моделей является класс rock: $mIoU/IoU$ у SegFormer ≈ 0.687 – 0.696 , что обусловлено высокой вариативностью внешнего вида скальных образований и меньшей долей класса в датасете.

7. Частоты кадров 13–14 FPS при разрешении 320×320 с использованием модели YOLOv8n-seg в формате NCNN является практически достаточной для решения задачи сегментации природных ориентиров и построения информационного вектора сцены в системе навигации БПЛА, при условии её интеграции с IMU-фильтром.

Проведённое исследование закладывает экспериментальную и методологическую основу для последующей разработки полноценной навигационной системы БПЛА, включая модуль сопоставления сцены с картой ориентиров, интеграцию с IMU через EKF-фильтр и тестирование в симуляторе AirSim.

Список используемой литературы

1. MidAir Dataset Official Website. University of Liège. [Электронный ресурс]. – Режим доступа: <https://midair.ulg.ac.be/> (дата обращения: 24.01.2026)
2. PyTorch Documentation. PyTorch Foundation. [Электронный ресурс]. – Режим доступа: <https://pytorch.org/docs/> (дата обращения: 24.01.2026)
3. OpenCV Documentation. Open Source Computer Vision Library. [Электронный ресурс]. – Режим доступа: <https://docs.opencv.org/> (дата обращения: 24.01.2026)
4. AirSim: Open source simulator for autonomous vehicles built on Unreal Engine. Microsoft Research. [Электронный ресурс]. – Режим доступа: <https://github.com/microsoft/AirSim> (дата обращения: 24.01.2026)
5. Real-Time Flying Object Detection with YOLOv8. [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/2305.09972> (дата обращения: 04.03.2026)
6. YOLOv11: An Overview of the Key Architectural Enhancements. [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/2410.17725> (дата обращения: 04.03.2026)
7. Ultralytics YOLO docs. [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/> (дата обращения: 04.03.2026)
8. Ultralytics/ultralytics YOLO. [Электронный ресурс]. – Режим доступа: <https://github.com/ultralytics/ultralytics> (дата обращения: 04.03.2026)
9. Models Supported by Ultralytics [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/models>. (Дата обращения 07.04.2026).
10. Xie E. et al. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/2105.15203> (дата обращения: 20.04.2026)
11. SegFormer. HuggingFace Transformers Documentation [Электронный ресурс]. – Режим доступа: https://huggingface.co/docs/transformers/model_doc/segformer (Дата обращения 20.04.2026).
12. Galoogahi H. K. et al. Need for Speed: A Benchmark for Higher Frame Rate Object Tracking // ICCV. — 2017. — [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/1703.05884> (дата обращения: 12.05.2026)
13. Ghanizadegan K., Hashim H. A. Unscented Particle Filter for Visual-Inertial Navigation using IMU and Landmark Measurements. — 2025. — [Электронный ресурс]. – Режим доступа: <https://arxiv.org/abs/2504.19318> (дата обращения: 12.05.2026)
14. Zhou X. et al. Visual SLAM for Unmanned Aerial Vehicles: Localization and Perception // PMC. — 2024. — [Электронный ресурс]. – Режим доступа: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11126069/> (дата обращения: 12.05.2026)

- 15 Lee J.-C. et al. Landmark-Based Scale Estimation and Correction of Visual Inertial Odometry for VTOL UAVs in a GPS-Denied Environment // *Sensors*. — 2022. — Vol. 22. — No. 24. — P. 9654. DOI: 10.3390/s22249654
- 16 Parra-Alonso I. et al. Real-time smart and standalone vision/IMU navigation sensor // *Journal of Real-Time Image Processing*. — 2016. — [Электронный ресурс]. — Режим доступа: <https://link.springer.com/article/10.1007/s11554-016-0613-z> (дата обращения: 12.05.2026)
- 17 Ma L. et al. Visual localization with a monocular camera for unmanned aerial vehicle based on landmark detection and tracking using YOLOv5 and DeepSORT // *International Journal of Advanced Robotic Systems*. — 2023. — [Электронный ресурс]. — Режим доступа: <https://journals.sagepub.com/doi/full/10.1177/17298806231164831> (дата обращения: 12.05.2026)
- 18 UAV detection with YOLO on a standalone Raspberry Pi 5 system // *CEUR Workshop Proceedings*. — Vol. 3970. — [Электронный ресурс]. — Режим доступа: <https://ceur-ws.org/Vol-3970/PAPER1.pdf> (дата обращения: 12.05.2026)
- 19 Lin X. et al. An Unmanned Aerial Vehicle Indoor Low-Computation Navigation Method Based on Vision and Deep Learning // *Sensors*. — 2024. — [Электронный ресурс]. — Режим доступа: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10781313/> (дата обращения: 12.05.2026)
- 20 Arjun R. et al. Learning Approach to Efficient Vision-based Active Tracking of a Flying Target by an Unmanned Aerial Vehicle. — 2025. — [Электронный ресурс]. — Режим доступа: <https://arxiv.org/html/2506.18264v1> (дата обращения: 12.05.2026)

Приложение А

```

import os
import json
import h5py
import numpy as np
from PIL import Image
import cv2
from collections import defaultdict

H5_PATH = r"F:\MidAir\Kite_training\sunny\sensor_records.hdf5"
BASE_DIR = os.path.dirname(H5_PATH)

TRAJ = "trajectory_0000"
MAX_FRAMES = 200      # сколько кадров анализировать
MIN_COMPONENT_AREA = 500 # отсечь мелкий шум
MAX_EXAMPLES_PER_LABEL = 12

OUT_DIR = "label_catalog"
os.makedirs(OUT_DIR, exist_ok=True)

def read_item_path(item):
    if isinstance(item, (bytes, bytearray)):
        return item.decode("utf-8").strip()
    raise TypeError(f"Unexpected item type: {type(item)}")

def load_image_from_h5_item(item, base_dir):
    rel_path = read_item_path(item)
    full_path = os.path.join(base_dir, rel_path)
    return np.array(Image.open(full_path))

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def main():
    with h5py.File(H5_PATH, "r") as f:
        seg_ds = f[f"{TRAJ}/camera_data/segmentation"]
        rgb_ds = f[f"{TRAJ}/camera_data/color_left"]
        depth_ds = f[f"{TRAJ}/camera_data/depth"]

        n = min(len(seg_ds), MAX_FRAMES)

        label_stats = defaultdict(lambda: {
            "pixel_count_total": 0,

```

```

"frames_seen": 0,
"center_x_sum": 0.0,
"center_y_sum": 0.0,
"depth_sum": 0.0,
"depth_count": 0,
"examples_saved": 0,
})

for idx in range(n):
    seg = load_image_from_h5_item(seg_ds[idx], BASE_DIR)
    rgb = load_image_from_h5_item(rgb_ds[idx], BASE_DIR)
    depth = load_image_from_h5_item(depth_ds[idx], BASE_DIR)

    labels = np.unique(seg)

    for label in labels:
        mask = (seg == label).astype(np.uint8)

        pixel_count = int(mask.sum())
        if pixel_count == 0:
            continue

        ys, xs = np.where(mask > 0)
        cx = float(xs.mean())
        cy = float(ys.mean())

        stat = label_stats[int(label)]
        stat["pixel_count_total"] += pixel_count
        stat["frames_seen"] += 1
        stat["center_x_sum"] += cx
        stat["center_y_sum"] += cy

        depth_vals = depth[mask > 0]
        if depth_vals.size > 0:
            stat["depth_sum"] += float(depth_vals.mean())
            stat["depth_count"] += 1

        # connected components: берём крупные экземпляры label
        num_labels, cc, cc_stats, _ = cv2.connectedComponentsWithStats(mask,
connectivity=8)

        for comp_id in range(1, num_labels):
            x, y, w, h, area = cc_stats[comp_id]
            if area < MIN_COMPONENT_AREA:
                continue

            if stat["examples_saved"] >= MAX_EXAMPLES_PER_LABEL:
                break

```



```

crop_rgb = rgb[y:y+h, x:x+w]
crop_mask = (cc[y:y+h, x:x+w] == comp_id).astype(np.uint8) * 255

# применим маску к RGB, чтобы было видно только объект
masked_rgb = crop_rgb.copy()
if masked_rgb.ndim == 3:
    masked_rgb[crop_mask == 0] = 0

label_dir = os.path.join(OUT_DIR, f"label_{int(label)}")
ensure_dir(label_dir)

Image.fromarray(masked_rgb).save(
    os.path.join(label_dir,
f"{TRAJ}_{idx:06d}_{stat['examples_saved']:02d}_rgb.png")
    )
Image.fromarray(crop_mask).save(
    os.path.join(label_dir,
f"{TRAJ}_{idx:06d}_{stat['examples_saved']:02d}_mask.png")
    )

stat["examples_saved"] += 1

# усредняем статистику
summary = {}
for label, stat in label_stats.items():
    summary[label] = {
        "pixel_count_total": stat["pixel_count_total"],
        "frames_seen": stat["frames_seen"],
        "avg_center_x": stat["center_x_sum"] / max(stat["frames_seen"], 1),
        "avg_center_y": stat["center_y_sum"] / max(stat["frames_seen"], 1),
        "avg_depth_raw": stat["depth_sum"] / max(stat["depth_count"], 1),
        "examples_saved": stat["examples_saved"],
    }

with open(os.path.join(OUT_DIR, "label_summary.json"), "w", encoding="utf-8") as
fp:
    json.dump(summary, fp, indent=2, ensure_ascii=False)

print("Done. Results saved to:", OUT_DIR)

if __name__ == "__main__":
    main()

```

Приложение Б

```

import os
import json
import h5py
import numpy as np
from PIL import Image
import cv2

H5_PATH = r"F:\MidAir\Kite_training\sunny\sensor_records.hdf5"
BASE_DIR = os.path.dirname(H5_PATH)

TRAJ = "trajectory_0000"

OUT_DIR = "yolo_dataset"
IMG_DIR = os.path.join(OUT_DIR, "images")
LBL_DIR = os.path.join(OUT_DIR, "labels")

os.makedirs(IMG_DIR, exist_ok=True)
os.makedirs(LBL_DIR, exist_ok=True)

LABEL_MAP_PATH = "label_catalog/label_map.json"

MIN_AREA = 400

def read_item_path(item):
    return item.decode("utf-8")

def load_image(item):
    path = os.path.join(BASE_DIR, read_item_path(item))
    return np.array(Image.open(path))

def mask_to_polygon(mask):
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    polygons = []

    for cnt in contours:
        if len(cnt) < 6:
            continue

        epsilon = 0.002 * cv2.arcLength(cnt, True)
        approx = cv2.approxPolyDP(cnt, epsilon, True)

        if len(approx) < 3:
            continue

```

```

        polygon = approx.reshape(-1, 2)
        polygons.append(polygon)

    return polygons

def normalize_polygon(poly, w, h):

    poly = poly.astype(float)

    poly[:, 0] /= w
    poly[:, 1] /= h

    return poly

def main():

    with open(LABEL_MAP_PATH) as f:
        label_map = json.load(f)

    classes = list(label_map.values())
    class_to_id = {c: i for i, c in enumerate(classes)}

    with h5py.File(H5_PATH, "r") as f:

        seg_ds = f[f"/{TRAJ}/camera_data/segmentation"]
        rgb_ds = f[f"/{TRAJ}/camera_data/color_left"]

        n = len(seg_ds)

        for idx in range(n):

            seg = load_image(seg_ds[idx])
            rgb = load_image(rgb_ds[idx])

            h, w = seg.shape

            img_name = f"{idx:06d}.png"

            Image.fromarray(rgb).save(os.path.join(IMG_DIR, img_name))

            label_lines = []

            for label, name in label_map.items():

                label = int(label)

```

```

class_id = class_to_id[name]

mask = (seg == label).astype(np.uint8)

if mask.sum() == 0:
    continue

num, cc, stats, _ = cv2.connectedComponentsWithStats(mask, connectivity=8)

for i in range(1, num):

    x, y, w_box, h_box, area = stats[i]

    # фильтр маленьких областей
    if area < MIN_AREA:
        continue

    # фильтр слишком маленьких bbox
    if w_box < 15 or h_box < 15:
        continue

    comp_mask = (cc == i).astype(np.uint8)

    polygons = mask_to_polygon(comp_mask)

    for poly in polygons:

        poly = normalize_polygon(poly, w, h)

        poly = poly.reshape(-1)

        line = str(class_id) + " " + " ".join(map(str, poly))

        label_lines.append(line)

    with open(os.path.join(LBL_DIR, img_name.replace(".png", ".txt")), "w") as ftxt:
        ftxt.write("\n".join(label_lines))

    if idx % 50 == 0:
        print("processed frame", idx)

if __name__ == "__main__":
    main()

```

Приложение В

```

import time
import torch
from ultralytics import YOLO

TEST_IMAGE = "yolo_dataset/images/val/000121.png"

models = {
    "yolov8n": "runs/segment/yolov8n-seg/weights/best.pt",
    "yolov8s": "runs/segment/yolov8s-seg/weights/best.pt",
    "yolov8m": "runs/segment/yolov8m-seg/weights/best.pt",
    "yolo11n": "runs/segment/yolo11n-seg/weights/best.pt",
    "yolo11s": "runs/segment/yolo11s-seg/weights/best.pt",
    "yolo11m": "runs/segment/yolo11m-seg/weights/best.pt",
}

for name, path in models.items():

    print("\nBenchmark:", name)

    model = YOLO(path)

    # warmup
    for _ in range(20):
        model.predict(TEST_IMAGE, imgsz=640, conf=0.25, device=0, verbose=False)

    torch.cuda.synchronize()

    times = []

    for _ in range(100):

        start = time.time()

        model.predict(TEST_IMAGE, imgsz=640, conf=0.25, device=0, verbose=False)

        torch.cuda.synchronize()

        end = time.time()

        times.append((end - start) * 1000)

    avg = sum(times) / len(times)

    fps = 1000 / avg

    print(f"avg inference: {avg:.2f} ms")
    print(f"FPS: {fps:.1f}")

```

Приложение Г

```

import os
import json
from multiprocessing import freeze_support

import numpy as np
from PIL import Image

import torch
import torch.nn.functional as F
from torch.utils.data import Dataset

import evaluate
from transformers import (
    SegformerImageProcessor,
    SegformerForSemanticSegmentation,
    TrainingArguments,
    Trainer,
)

DATASET_DIR = "segformer_dataset"
NUM_CLASSES = 4 # background, tree, rock, river

MODEL_NAMES = [
    "nvidia/segformer-b0-finetuned-ade-512-512",
    "nvidia/segformer-b1-finetuned-ade-512-512",
    "nvidia/segformer-b2-finetuned-ade-512-512",
]

OUTPUT_ROOT = "segformer_runs"

class MidAirSegDataset(Dataset):
    def __init__(self, root_dir, split, processor):
        self.image_dir = os.path.join(root_dir, "images", split)
        self.mask_dir = os.path.join(root_dir, "masks", split)
        self.processor = processor

        self.files = sorted([
            x for x in os.listdir(self.image_dir)
            if x.lower().endswith(".png")
        ])

    def __len__(self):
        return len(self.files)

    def __getitem__(self, idx):
        name = self.files[idx]

```



```

image = Image.open(os.path.join(self.image_dir, name)).convert("RGB")
mask = Image.open(os.path.join(self.mask_dir, name))
mask = np.array(mask, dtype=np.uint8)

encoded = self.processor(
    images=image,
    segmentation_maps=mask,
    return_tensors="pt"
)

return {
    "pixel_values": encoded["pixel_values"].squeeze(0),
    "labels": encoded["labels"].squeeze(0).long(),
}

def collate_fn(batch):
    pixel_values = torch.stack([x["pixel_values"] for x in batch])
    labels = torch.stack([x["labels"] for x in batch])
    return {
        "pixel_values": pixel_values,
        "labels": labels,
    }

def make_compute_metrics(num_labels):
    metric = evaluate.load("mean_iou")

    def compute_metrics(eval_pred):
        logits, labels = eval_pred
        logits = torch.from_numpy(logits)

        logits = F.interpolate(
            logits,
            size=labels.shape[-2:],
            mode="bilinear",
            align_corners=False,
        )

        pred_labels = logits.argmax(dim=1).cpu().numpy()

        metrics = metric.compute(
            predictions=pred_labels,
            references=labels,
            num_labels=num_labels,
            ignore_index=255,
            reduce_labels=False,
        )

```

```

per_iou = metrics.pop("per_category_iou", None)
per_acc = metrics.pop("per_category_accuracy", None)

if per_iou is not None:
    for i, v in enumerate(per_iou):
        metrics[f"iou_class_{i}"] = float(v)

if per_acc is not None:
    for i, v in enumerate(per_acc):
        metrics[f"acc_class_{i}"] = float(v)

return {k: float(v) for k, v in metrics.items()}

return compute_metrics

def train_one_model(model_name):
    short_name = model_name.split("/")[-1]
    out_dir = os.path.join(OUTPUT_ROOT, short_name)

    id2label = {
        0: "background",
        1: "tree",
        2: "rock",
        3: "river",
    }
    label2id = {v: k for k, v in id2label.items()}

    processor = SegformerImageProcessor(
        do_resize=True,
        size={"height": 512, "width": 512},
        do_reduce_labels=False,
    )

    train_ds = MidAirSegDataset(DATASET_DIR, "train", processor)
    val_ds = MidAirSegDataset(DATASET_DIR, "val", processor)

    model = SegformerForSemanticSegmentation.from_pretrained(
        model_name,
        num_labels=NUM_CLASSES,
        id2label=id2label,
        label2id=label2id,
        ignore_mismatched_sizes=True,
    )

    args = TrainingArguments(
        output_dir=out_dir,
        learning_rate=6e-5,
        num_train_epochs=20,

```

```

    per_device_train_batch_size=2,
    per_device_eval_batch_size=1,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_strategy="epoch",
    save_total_limit=2,
    load_best_model_at_end=True,
    metric_for_best_model="mean_iou",
    greater_is_better=True,
    remove_unused_columns=False,
    dataloader_num_workers=0,
    eval_accumulation_steps=1,
    report_to="none",
    fp16=torch.cuda.is_available(), # экономит память на NVIDIA
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    processing_class=processor,
    data_collator=collate_fn,
    compute_metrics=make_compute_metrics(NUM_CLASSES),
)

trainer.train()

eval_metrics = trainer.evaluate()
print(f"\nFinal metrics for {short_name}:")
print(eval_metrics)

best_dir = os.path.join(out_dir, "best_model")
trainer.save_model(best_dir)
processor.save_pretrained(best_dir)

with open(os.path.join(out_dir, "eval_results.json"), "w", encoding="utf-8") as f:
    json.dump(eval_metrics, f, indent=2, ensure_ascii=False)

def main():
    for model_name in MODEL_NAMES:
        print(f"\n=== Training {model_name} ===")
        torch.cuda.empty_cache()
        train_one_model(model_name)

if __name__ == "__main__":
    freeze_support()
    main()

```